

コンパイラ (13)

2025/11/14

静的単一代入を用いた最適化

データフロー解析のもうひとつの手法:

静的単一代入 (Static Single Assignment: SSA)

各変数の定義がプログラム中で唯一

二度目の代入が発生すると変数名を変更

値が同じの共通部分式が判別できる=式として一致

定義ごとに変数に添字をつける。合流する場合は、 ϕ 関数で表現
いかにして、SSA 形式を得るか？

- (1) ϕ 関数の挿入
- (2) 変数の添字の決定

SSA 形式において最適化

- (3) SSA から逆変換する。(ϕ 関数の消去)

近年のコンパイラでよく用いられる手法

例

```
scanf("%d",&x);
a=1;
b=2;
c=a+b;
if(x>0) a=3;
d=b;
c=a+b;
```

⇒

```
scanf("%d",&x0);
a0=1;
b0=2;
c0=a0+b0;
if(x>0) a1=3;
a2= $\phi$ (a0,a1);
d0=b0;
c1=a2+b0;
```

```
t=1
n=10
l1: t=t*n
n=n-1
b=n>1
if b goto l1
f=t
```

⇒

```
t1=1
n1=10
l1: t2= $\phi$ (t1,t3)
n2= $\phi$ (n1,n3)
t3=t2*n2
n3=n2-1
b1=n3>1
if b1 goto l1
f1=t3
```

ϕ 関数の挿入

根つきグラフ: (V, E)

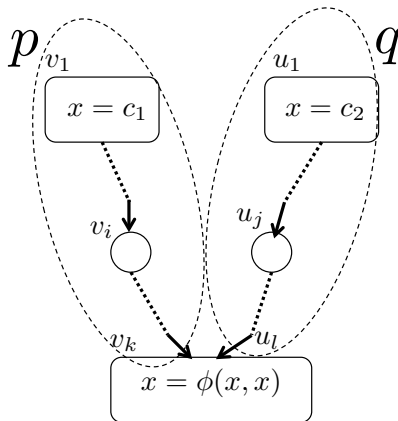
先行頂点を持たない唯一の接点 v_o (根) が存在

定義 (定義合流条件)

変数 x の ϕ 関数が頂点 v に挿入されるための必要十分条件は、次の条件 (1)~(4) を満たす 2 つのパス $p: V_1, \dots, v_k$ および $q: u_1, \dots, u_\ell$ が存在することである。

- (1) $k > 1, \ell > 1$
- (2) v_1, u_1 の両方に x の定義が存在
- (3) $v_k = u_\ell = v$
- (4) $\{v_1, \dots, v_{k-1}\} \cap \{u_1, \dots, u_{\ell-1}\} = \emptyset$

定義合流条件



支配関係

定義 (頂点間の支配関係)

フローグラフ $G = (V, E)$ が与えられたとき, $u, v \in V$ に対して, 開始頂点 v_0 から v に至るすべてのパス $p: v_0, \dots, v_k$ (ここで $v = v_k$) において, $u \in \{v_0, \dots, v_k\}$ であるとき, u は v を支配するといい, $u \gg v$ と書く.

命題

$\gg$ は, 反射率 ($u \gg u$) および推移律 ($u \gg v$ かつ $v \gg w$ ならば $u \gg w$) を満たす.

$u \gg v$ かつ $u \neq v$ のとき, u は v を強く支配するといい, $u \ggg v$ と書く.

$Dom(v) = \{u \mid u \gg v\}$ を v は支配頂点の集合を表す.

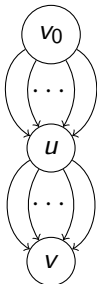
支配関係

- ▶ $Dom(v) \neq \emptyset$
- ▶ $Dom(v) \setminus \{v\} \neq \emptyset$ のとき、唯一の $u \in Dom(v) \setminus \{v\}$ が存在して、すべての $w \in Dom(v) \setminus \{v\}$ に対して、 $w \geq u$ である。
(唯一性の証明) $u_1, u_2 \in Dom(v) \setminus \{v\}$ であるならば、開始頂点から v へすべてのパス上に u_1, u_2 が存在する。よって、 $u_1 \geq u_2$ または $u_2 \geq u_1$ である。したがって、 $u_1 = u_2$ となる。このような u を $idom(v)$ と書く。

(支配木)

有向グラフ (V, E') : $E = \{(idom(v), v) \mid v \in V\}$ は、木構造となり、これを支配木とよぶ。

支配関係, 支配木



$$u \gg v$$

$$u \gg u$$

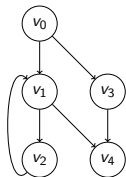
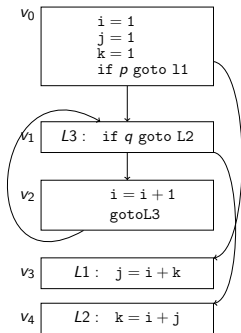
$$u \gg v, v \gg w \text{ ならば } u \gg w$$

$$u \neq v \text{ のとき } u \gg v$$

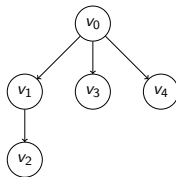
v 以外で最も下にある支配頂点 u : $idom(v)$

$(idom(v), v)$ を辺とするグラフ = **支配木**

フローグラフ → 支配木



フローグラフ



支配木

支配辺境

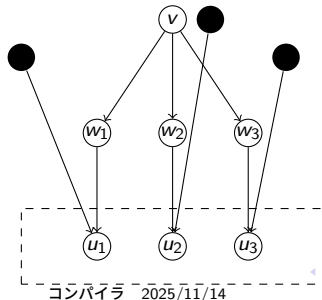
フローグラフ $G = (V, E)$

定義

v の支配辺境 (*Dominance Frontier*)

$$DF(v) = \{u \in V \mid \exists w. w \in pred(u), v \geq w, v \not\geq u\}$$

強く支配していれば ϕ 関数を挿入する必要なし



支配境界の計算

フローグラフ $G = (V, E)$

補題

$$DF(v) = DF_{local}(v) \cup \bigcup_{w \in Child(v)} DF_{up}^v(w)$$

ここで、

$DF_{local}(v) = \{u \mid (v, u) \in E, \text{idom}(u) \neq v\}$
 v の次の頂点で他からのフローが存在する

$DF_{up}^v(w) = \{u \in DF(w) \mid \text{idom}(u) \neq v\}$
 w は支配木において v の子で u から見て、
 v は w よりも上 (根に近い) にある。

支配木の葉から DF を計算していく (p129: 例)

定義合流条件の計算

定義合流条件 ($\mathcal{J}$) 二つのパス $p(= v_1, \dots, v_k)$, $q(= u_1, \dots, u_l)$

$$(1) \quad k \geq 2, l \geq 2$$

$$(2) \quad v_1, u_1 \in S$$

$$(3) \quad v_k = u_l = v$$

$$(4) \quad \{v_1, \dots, v_{k-1}\} \cap \{u_1, \dots, u_{l-1}\} = \emptyset$$

$$J(S) = \{v \mid \text{上の条件を満たす } v\}$$

変数 x が定義される場所の集合を S とすると $J(S)$ は、 x に対して最初に ϕ 関数 $x = \phi(x, x)$ が挿入される場所

ϕ 関数によって新たに定義が挿入されるため、さらに挿入が必要: 次の漸化式で不動点 $J^+(S_0)$ を計算

$$\begin{aligned} S_{i+1} &= J(S_0 \cup S_i) \\ J^{n+1}(S_0) &= J(S_0 \cup J^n(S_0)) \\ J^+(S_0) &= \lim_{n \leftarrow \infty} J^n(S_0) \end{aligned}$$

支配辺境と合流条件

定理

$DF(S) = \bigcup_{v \in S} DF(v)$ のとき、

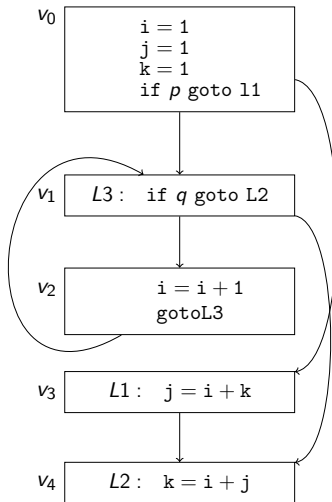
$$F_{i+1} = DF(S_0 \cup F_i) \quad (i \geq 0, F_0 = \emptyset)$$

とするとき、 $DF^+(S_0) = \lim_{n \leftarrow \infty} F_n$ とする。このとき、

$$DF^+(S_0) = J^+(S_0)$$

合流条件は支配辺境から計算可能

例 6.29



i の定義: $S_0 = \{v_0, v_2\}$

j の定義: $S'_0 = \{v_0, v_3\}$

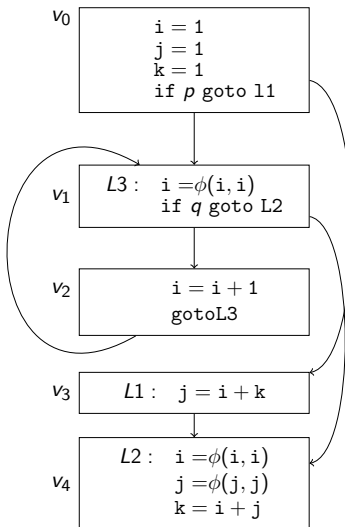
k の定義: $S''_0 = \{v_0, v_4\}$

$DF^+(S_0) = \{v_1, v_4\}$

$DF^+(S'_0) = \{v_4\}$

$DF^+(S''_0) = \emptyset$

例 6.29



i の定義: $S_0 = \{v_0, v_2\}$

j の定義: $S'_0 = \{v_0, v_3\}$

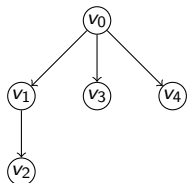
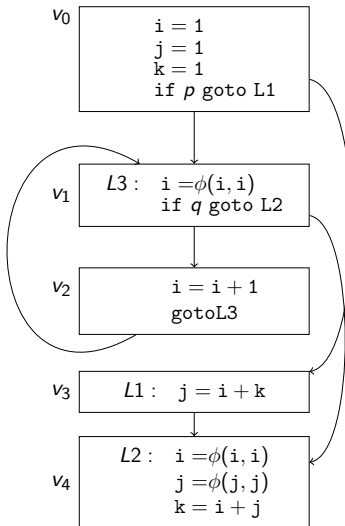
k の定義: $S''_0 = \{v_0, v_4\}$

$DF^+(S_0) = \{v_1, v_4\}$

$DF^+(S'_0) = \{v_4\}$

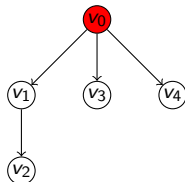
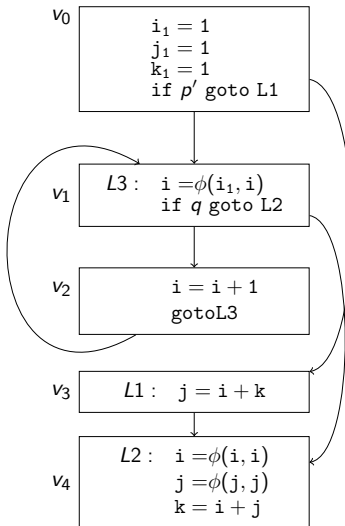
$DF^+(S''_0) = \emptyset$

SSA 形式への変換



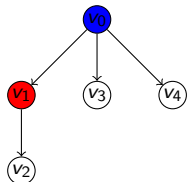
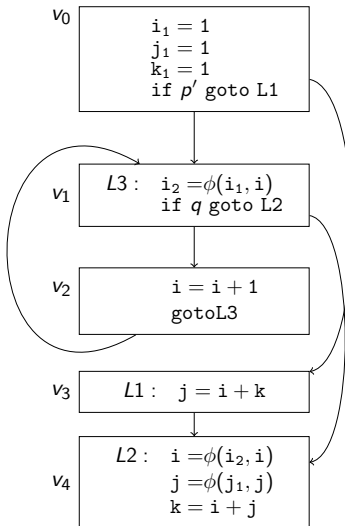
$stack(i) = [] \quad c(i) = 1$
 $stack(j) = [] \quad c(j) = 1$
 $stack(k) = [] \quad c(k) = 1$

SSA 形式への変換



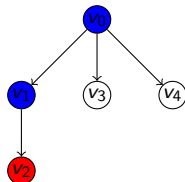
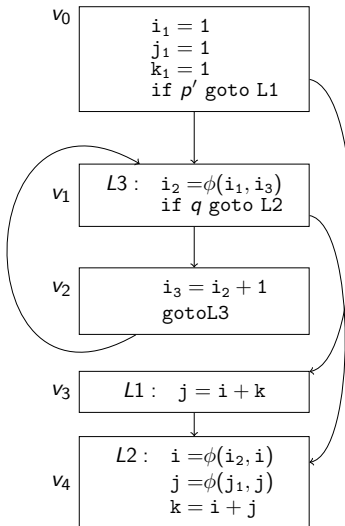
$stack(i) = [1] \quad c(i) = 2$
 $stack(j) = [1] \quad c(j) = 2$
 $stack(k) = [1] \quad c(k) = 2$

SSA 形式への変換



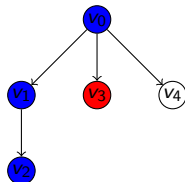
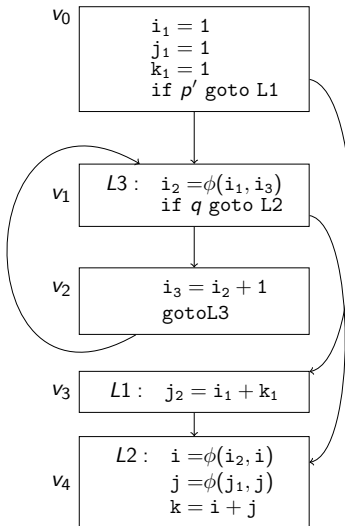
$stack(i) = [1, 2] \quad c(i) = 3$
 $stack(j) = [1] \quad c(j) = 2$
 $stack(k) = [1] \quad c(k) = 2$

SSA 形式への変換



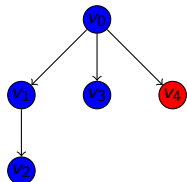
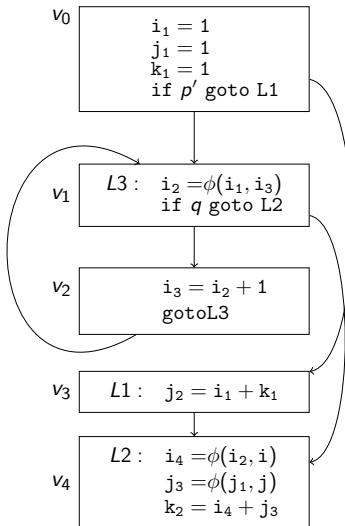
$stack(i) = [1, 2, 3]$ $c(i) = 4$
 $stack(j) = [1]$ $c(j) = 2$
 $stack(k) = [1]$ $c(k) = 2$

SSA 形式への変換



$stack(i) = [1, 2, 3]$ $c(i) = 4$
 $stack(j) = [1, 2]$ $c(j) = 3$
 $stack(k) = [1]$ $c(k) = 2$

SSA 形式への変換



$stack(i) = [1, 2, 3]$ $c(i) = 4$
 $stack(j) = [1, 2]$ $c(j) = 3$
 $stack(k) = [1, 2]$ $c(k) = 3$

SSA 形式に対する最適化

SSA 形式：データフロー解析ずみのプログラム

構文的にフローが表現されている ⇒ 最適化を効果的に適用できる：更新のチェックが不要

例：DAG の構成が容易

- ▶ 定数伝播
- ▶ コピー伝播
- ▶ 共通部分式の削除
- ▶ ループ不変コードの移動
- ▶ 不用コードの削除

SSA 最適化例 [佐々:06]

	$a_1=1;$
$a=1;$	$b_1=2;$
$b=2;$	$x_1=3;$
$x=3;$	$y_1=a_1+b_1;$
$y=a+b;$	$c_1=x_1+y_1;$
$c=x+y;$	$\text{if}(a_1<10)$
$\text{if}(a<10)$	$\quad z_1=a_1+b_1;$
$\quad z=a+b;$	else
else	$\quad z_2=a_1+b_1;$
$\quad z=a+b;$	$z_3=\phi(z_1, z_2);$
$d=x+z;$	$d_1=x_1+z_3;$
$\text{println}(d);$	$\text{println}(d_1);$

$\Rightarrow$

SSA 形式へ変換

SSA 最適化例 [佐々:06]

<pre>a₁=1; b₁=2; x₁=3; y₁=a₁+b₁; c₁=x₁+y₁; if (a₁<10) z₁=a₁+b₁; else z₂=a₁+b₁; z₃=ϕ(z₁, z₂); d₁=x₁+z₃; println(d₁);</pre>	$\Rightarrow$	<pre>a₁=1; b₁=2; x₁=3; y₁=a₁+b₁; c₁=x₁+y₁; if (a₁<10) z₁=y₁; else z₂=y₁; z₃=y₁ (= ϕ(y₁, y₁)); d₁=x₁+z₃; println(d₁);</pre>
---	---------------	---

共通部分式除去

SSA 最適化例 [佐々:06]

<pre>a₁=1; b₁=2; x₁=3; y₁=a₁+b₁; c₁=x₁+y₁; if (a₁<10) z₁=y₁; else z₂=y₁; z₃=y₁(=ϕ(y₁,y₁)); d₁=x₁+z₃; println(d₁);</pre>	$\Rightarrow$	<pre>a₁=1; b₁=2; x₁=3; y₁=a₁+b₁; c₁=x₁+y₁; if (a₁<10) z₁=y₁; else z₂=y₁; z₃=y₁; d₁=x₁+y₁; println(d₁);</pre>
--	---------------	---

コピー伝播

SSA 最適化例 [佐々:06]

<code>a₁=1;</code>		<code>a₁=1;</code>
<code>b₁=2;</code>		<code>b₁=2;</code>
<code>x₁=3;</code>		<code>x₁=3;</code>
<code>y₁=a₁+b₁;</code>		<code>y₁=3;</code>
<code>c₁=x₁+y₁;</code>		<code>c₁=6;</code>
<code>if (a₁<10)</code>		<code>if (a₁<10)</code>
<code>z₁=y₁;</code>		<code>z₁=y₁;</code>
<code>else</code>		<code>else</code>
<code>z₂=y₁;</code>		<code>z₂=y₁;</code>
<code>z₃=y₁;</code>		<code>z₃=y₁;</code>
<code>d₁=x₁+y₁;</code>		<code>d₁=c₁;</code>
<code>println(d₁);</code>	$\Rightarrow$	<code>println(d₁);</code>

共通部分式除去、定数伝播

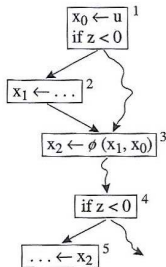
SSA 最適化例 [佐々:06]

```
a1=1;  
b1=2;  
x1=3;  
y1=3;  
c1=6;  
if(a1<10)           a1=1;  
    z1=y1;           b1=2;  
else                 x1=3;  
    z2=y1;           y1=3;  
z3=y1;             c1=6;  
d1=c1;             z3=y1;  
println(d1);        println(6);  
⇒
```

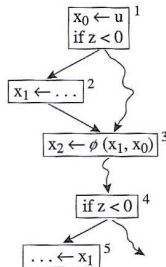
コピー伝播、定数伝播、恒等式

最適化の条件

最適化を行うときに支配関係を壊さないようにする。



(a) Before



(b) After

$z < 0$ の評価は同じ方向にしかブランチしない

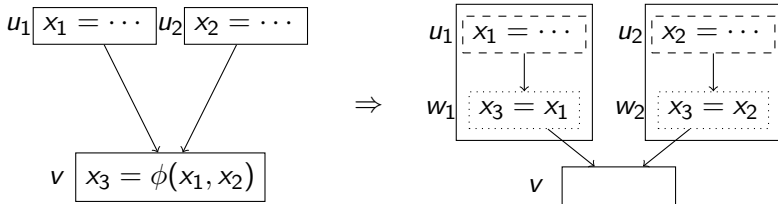
$\Rightarrow z < 0$ ならば 3 の ϕ 関数はいつも x_1 を x_2 へ代入

5 において $z < 0$ で、 x_1 を参照に用いると支配関係が壊れる

(支配関係を壊すと一般にはデータフローが保証されない)

静的と動的のギャップを ϕ 関数で埋めているため

SSA 形式からの逆変換： ϕ 関数の消去



ϕ 関数に対してはコード生成ができない。

(先の例はたまたま ϕ 関数まで消えてしまった。)

ノード w_1, w_2 を挿入して、 ϕ 関数を消去

3 番地コード $\rightarrow$ SSA 形式 $\rightarrow$ 最適化 $\rightarrow$ ϕ 関数消去

逆変換アルゴリズム

1. フローグラフの変換

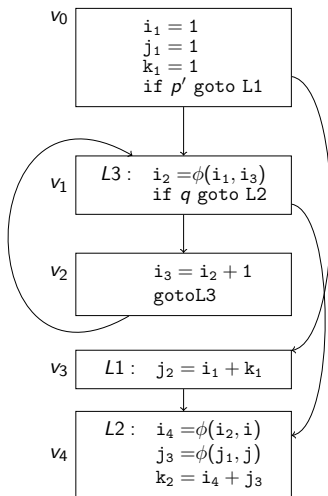
ϕ 関数を持つブロックに対して挿入条件を満たさないまで、代入文を挿入する。

挿入条件： ϕ 関数を持つブロックが複数の先行頂点を持ち、そのうちの 하나가複数の後続頂点を持つ。

先行ブロックの最後が goto 文の場合は、goto 文の直前に代入文を挿入する。

2. ϕ 関数の消去

逆変換例



逆変換例

